

UNIVERSIDAD PRIVADA ANTENOR ORREGO
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E
INTELIGENCIA ARTIFICIAL



PROYECTO DE INTELIGENCIA ARTIFICIAL

Detección de COVID-19 en Radiografías de Tórax mediante Deep Learning

Curso: INTELIGENCIA ARTIFICIAL: PRINCIPIOS Y TÉCNICAS

Código: ISIA-108 | **Ciclo:** 06 | **Semestre:** 2026-10

Estudiante: Torres Ruiz, Ángel Eduardo

Docente: Dr. Teobaldo Hernán Sagástegui

Fecha y Lugar: Trujillo, mayo del 2026

Contenido

I. INTRODUCCIÓN.....	3
1.1 Título del Proyecto.....	3
1.2 Antecedentes.....	3
1.3 Problema a Resolver	4
1.4 Objetivos.....	4
II. REQUERIMIENTOS.....	5
2.1 Definición del Dominio	5
2.2 Determinación de Requisitos (Requerimientos).....	5
III. PRE-PROCESAMIENTO Y NORMALIZACIÓN	6
3.1 Medidas, Datos, Bases de Datos y Elaboración del Data-Set.....	6
Fuentes de Datos:.....	6
Características de las Imágenes:.....	6
Elaboración del Dataset Consolidado:	7
3.2 Normalización y/o Filtrado de Datos	7
Pipeline de Preprocesamiento:	8
3.3 Planteamiento de la Organización del Data-Set	8
IV. APRENDIZAJE.....	10
4.1 Planteamiento del Modelo de Aprendizaje.....	10
Tipo de Aprendizaje:.....	10
Arquitecturas CNN Consideradas:	10
Estrategia de Transfer Learning:	10
Función de Pérdida y Optimizador:.....	11
Regularización y Técnicas Anti-Sobreajuste:.....	11
4.2 Desarrollo e Implementación del Modelo	12
Entorno de Desarrollo:.....	12
Fases de Implementación:	12
Métricas de Evaluación:.....	15
REFERENCIAS BIBLIOGRÁFICAS	16

I. INTRODUCCIÓN

1.1 Título del Proyecto

Detección y Clasificación de COVID-19 en Radiografías de Tórax mediante Redes Neuronales Convolucionales y Técnicas de Deep Learning.

El proyecto se desarrolla a partir del notebook de Kaggle «Intro IA 3.0-DL Imagen-4-Covid-19» (usuario fizcogar), que implementa un modelo de clasificación de imágenes de rayos X de tórax para distinguir entre pacientes con COVID-19, neumonía común y personas sanas. Este trabajo se enmarca en la asignatura de Inteligencia Artificial: Principios y Técnicas (ISIA-108), cubriendo contenidos de las semanas 10 a 13 del sílabo: Deep Learning, Redes Neuronales Convolucionales, y Visión por Computadora.

1.2 Antecedentes

La pandemia de COVID-19 (causada por el virus SARS-CoV-2) generó una crisis sanitaria global sin precedentes. La detección temprana y precisa fue esencial para contener la propagación del virus. Las pruebas PCR, si bien son el estándar de oro, presentan tiempos de espera prolongados y tasas de falsos negativos de hasta un 30%. Las radiografías de tórax (CXR) constituyen una alternativa rápida, económica y ampliamente disponible para la detección de hallazgos pulmonares compatibles con COVID-19, tales como opacidades en vidrio esmerilado, consolidaciones bilaterales y patrones de neumonía atípica.

En este contexto, diversos estudios demostraron la viabilidad de utilizar Deep Learning, particularmente Redes Neuronales Convolucionales (CNN), para la clasificación automática de radiografías de tórax. Trabajos como CheXNet (Rajpurkar et al., 2017), que entrenó una DenseNet-121 sobre el dataset ChestX-ray14, alcanzaron niveles de detección de neumonía comparables a radiólogos expertos. Posteriormente, durante la pandemia, modelos como COVID-Net y DarkCovidNet aplicaron transfer learning con arquitecturas ResNet, VGG16, DenseNet e InceptionV3 para clasificar imágenes de CXR en categorías de COVID-19, neumonía y normal, obteniendo precisión superior al 90% en múltiples estudios.

1.3 Problema a Resolver

El problema central consiste en la clasificación automática de radiografías de tórax en tres categorías: COVID-19, neumonía (viral/bacteriana) y normal/sano. Este es un problema de clasificación multiclase de imágenes que requiere:

- Distinguir patrones visuales sutiles entre COVID-19 y neumonía convencional, dado que ambas condiciones presentan hallazgos radiológicos similares (opacidades, consolidaciones).
- Manejar el desbalance de clases inherente a los datasets disponibles, donde las imágenes de COVID-19 suelen ser significativamente menos numerosas.
- Lograr alta sensibilidad para minimizar falsos negativos (pacientes enfermos no detectados) sin sacrificar excesivamente la especificidad.
- Desarrollar un modelo que sea generalizable y no sobreajuste a características específicas de un único hospital o equipo de rayos X.

1.4 Objetivos

Objetivo General: Desarrollar e implementar un modelo de Deep Learning basado en Redes Neuronales Convolucionales capaz de clasificar radiografías de tórax en las categorías COVID-19, neumonía y normal, alcanzando métricas de rendimiento competitivas ($\text{accuracy} \geq 90\%$, $\text{AUC-ROC} \geq 0.95$).

Objetivos Específicos:

- Construir y preprocesar un dataset de imágenes de radiografías de tórax a partir de repositorios públicos, aplicando técnicas de normalización y aumento de datos.
- Implementar arquitecturas CNN (tales como ResNet50, VGG16, DenseNet121, InceptionV3) utilizando transfer learning con pesos pre-entrenados en ImageNet.
- Entrenar, validar y comparar el rendimiento de distintos modelos utilizando métricas de clasificación (accuracy, precision, recall, F1-score, AUC-ROC).
- Aplicar técnicas de visualización (Grad-CAM) para interpretar las regiones de interés que el modelo identifica como indicativas de cada patología.
- Documentar el proceso completo conforme a la metodología de proyectos de IA establecida en la asignatura.

II. REQUERIMIENTOS

2.1 Definición del Dominio

Dominio de aplicación: Imagenología médica / Radiología torácica. El dominio del proyecto se sitúa en la intersección de la inteligencia artificial y la salud pública. Específicamente, se trabaja con imágenes médicas de tipo radiografía de tórax (Chest X-Ray, CXR), una de las modalidades de imagenología más utilizadas globalmente por su bajo costo, rapidez de adquisición y disponibilidad en centros de salud de todos los niveles. El problema se enmarca dentro del área de Visión por Computadora (Semana 13 del sílabo), utilizando técnicas de Deep Learning (Semana 10), específicamente Redes Neuronales Convolucionales (CNNs) con arquitecturas como AlexNet, VGG, GoogLeNet (Semana 10), y Mecanismos de Atención (Semana 11). El proyecto integra además conceptos de Machine Learning supervisado (Semana 9) para la tarea de clasificación. Las patologías objetivo son: COVID-19 (opacidades bilaterales en vidrio esmerilado, consolidaciones periféricas), neumonía viral/bacteriana (infiltrados alveolares, broncograma aéreo), y la condición normal (campos pulmonares sin hallazgos patológicos significativos).

2.2 Determinación de Requisitos (Requerimientos)

Requerimientos Funcionales:

- RF01: El sistema debe recibir como entrada una imagen de radiografía de tórax en formato estándar (JPEG, PNG) y devolver una clasificación entre las categorías: COVID-19, neumonía o normal.
- RF02: El modelo debe proporcionar las probabilidades asociadas a cada clase (softmax output), permitiendo al usuario evaluar la confianza de la predicción.
- RF03: El sistema debe generar mapas de activación (Grad-CAM) que visualicen qué regiones de la radiografía fueron determinantes para la predicción.
- RF04: El sistema debe ser capaz de procesar múltiples imágenes de manera batch y retornar resultados en tiempo razonable (< 2 segundos por imagen).
- RF05: El usuario debe contar con un espacio privado donde pueda gestionar su perfil y consultar diagnósticos realizados anteriormente.
- RF06: El usuario debe poder descargar un reporte con el diagnóstico y la imagen con el mapa de calor para su uso externo

Requerimientos No-Funcionales:

- RNF01: Accuracy mínima: 90% en el conjunto de pruebas balanceado.
- RNF02: AUC-ROC: ≥ 0.95 para la curva ROC multi-clase.
- RNF03: Sensibilidad (Recall) para COVID-19: $\geq 90\%$, minimizando falsos negativos.
- RNF04: El modelo debe ejecutarse en entornos con GPU limitada (NVIDIA T4 con 16 GB VRAM).
- RNF05: El código debe estar documentado y utilizar buenas prácticas de ingeniería de software.
- RNF06: El código debe ser entregado en formato notebook (.ipynb), estar documentado y seguir la estructura de referencia de los datasets de Kaggle proporcionados.
- RNF07: El sistema debe asegurar que los resultados del usuario se guarden correctamente en su historial personal dentro de la base de datos.

III. PRE-PROCESAMIENTO Y NORMALIZACIÓN

3.1 Medidas, Datos, Bases de Datos y Elaboración del Data-Set

El dataset utilizado en el proyecto se construye a partir de repositorios públicos de radiografías de tórax disponibles en Kaggle y otras fuentes académicas. Siguiendo el enfoque del notebook de referencia, se trabaja con un dataset de imágenes de CXR organizado en tres clases: COVID-19, Neumonía y Normal.

Fuentes de Datos:

- COVID-19 Radiography Database (Kaggle): Dataset público con miles de radiografías de tórax en formato JPEG, organizadas por clase (COVID-19, Neumonía, Normal).
- ChestX-ray14 (NIH): Dataset de 112,120 radiografías de tórax con anotaciones de múltiples enfermedades, incluyendo infiltrados compatibles con neumonía.
- Datos Sintéticos: Generación de imágenes adicionales mediante data augmentation para equilibrar clases minoritarias.

Características de las Imágenes:

Las imágenes originales tienen resoluciones variables (desde 256×256 hasta 2048×2048 píxeles). Para la alimentación a los modelos CNN preentrenados, todas

las imágenes se redimensionan a 224×224 píxeles (estándar de ImageNet para ResNet, VGG, DenseNet) o 299×299 píxeles (para InceptionV3). Las imágenes pueden estar en escala de grises (1 canal) o RGB (3 canales); en el pipeline se convierten a RGB de 3 canales para compatibilidad con los modelos preentrenados.

Elaboración del Dataset Consolidado:

Siguiendo la metodología del notebook de Kaggle, se seleccionan imágenes de las fuentes mencionadas y se organizan en una estructura de carpetas por clase. Se realiza un muestreo para balancear las clases cuando es necesario (oversampling de la clase minoritaria o undersampling de la mayoritaria), garantizando que el modelo no esté sesgado hacia la clase más frecuente. Se verifica la calidad visual de las imágenes, descartando aquellas con artefactos excesivos, rotaciones anormales o baja calidad diagnóstica.

3.2 Normalización y/o Filtrado de Datos

El preprocesamiento de las imágenes sigue un pipeline estándar de Visión por Computadora, implementado mediante la clase ImageDataGenerator de Keras (conforme a las herramientas del sílabo):

Redimensionamiento (Resize): Todas las imágenes se redimensionan a dimensiones fijas (224×224 o 299×299 píxeles según la arquitectura). Se utiliza interpolación bilineal para preservar la calidad visual.

Normalización de Píxeles: Los valores de píxeles se normalizan al rango $[0, 1]$ mediante división por 255 ($\text{rescale}=1./255$). Alternativamente, para modelos preentrenados en ImageNet, se aplica la normalización específica del modelo (`preprocess_input`).

Aumento de Datos (Data Augmentation): Para incrementar la robustez del modelo y combatir el sobreajuste, se aplican transformaciones aleatorias en tiempo de entrenamiento: rotación ($\pm 15^\circ$), desplazamiento horizontal y vertical ($\pm 10\%$), zoom ($\pm 10\%$), volteo horizontal (flip), y ajustes leves de brillo.

Filtrado de Calidad: Se eliminan imágenes duplicadas, imágenes laterales (solo se conservan proyecciones PA y AP), imágenes con anotaciones superpuestas (flechas, texto) que podrían introducir sesgo, e imágenes con resolución inferior al mínimo requerido.

Pipeline de Preprocesamiento:

Etapas	Operación	Parámetros	Herramienta
1. Carga	Lectura de imagen	Formato: JPEG/PNG	OpenCV
2. Resize	Redimensionamiento	224×224 px	PIL/OpenCV
3. Normalización	Escalado de píxeles	rescale = 1./255	Keras/NumPy
4. Augmentation	Transformaciones aleatorias	rot=15°, shift=10%	ImageDataGenerator
5. Batch	Agrupación en lotes	batch size = 32	Keras

```
# Normalización
normalization_layer = tf.keras.layers.Rescaling(1./255)
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))
```

3.3 Planteamiento de la Organización del Data-Set

La organización del dataset sigue la convención estándar de entrenamiento-prueba-validación. El dataset se divide en tres particiones con propósitos específicos:

Data-Set de Entrenamiento (Training) - 80%: Contiene el 80% de las imágenes totales y se utiliza para el aprendizaje de los parámetros del modelo (pesos de la red neuronal). Es sobre este conjunto donde se aplican las técnicas de aumento de datos (data augmentation). El entrenamiento se realiza mediante descenso de gradiente con el optimizador Adam (learning rate = 0.0001), utilizando categorical crossentropy como función de pérdida.

Data-Set de Pruebas (Test) - 10%: Contiene el 10% de las imágenes y se reserva exclusivamente para la evaluación final del modelo. Este conjunto no se utiliza durante el entrenamiento ni la validación; su propósito es proporcionar una estimación imparcial del rendimiento del modelo en datos completamente nuevos.

Data-Set de Validación (Validation/Cross-Validation) - 10%: Contiene el 10% de las imágenes y se utiliza durante el entrenamiento para monitorear el rendimiento del modelo en cada época (epoch). Se puede implementar K-Fold Cross-Validation (k=5) para obtener estimaciones más robustas.

Distribución del Data-Set:

Partición	COVID-19	Neumonía	Normal	Propósito
Training (80%)	~3,200	~3,200	~3,200	Aprendizaje
Validation (10%)	~400	~400	~400	Monitoreo
Test (10%)	~400	~400	~400	Evaluación
Total (~100%)	~4,000	~4,000	~4,000	~12,000 imág.

```
import tensorflow as tf
import os

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# Ruta a tu nueva estructura unificada
ruta_train = '/content/drive/MyDrive/DATASET_PULMONES/train'
ruta_test = '/content/drive/MyDrive/DATASET_PULMONES/test'

print("Cargando Dataset de Entrenamiento (3 Clases)...")
train_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_train,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="training",
    seed=42
)

print("\nCargando Dataset de Validación...")
val_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_train,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="validation",
    seed=42
)

print("\nCargando Dataset de Pruebas (Test)...")
test_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_test,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False # Importante para la Matriz de Confusión
)
```

IV. APRENDIZAJE

4.1 Planteamiento del Modelo de Aprendizaje

Tipo de Aprendizaje:

El proyecto utiliza Aprendizaje Supervisado mediante técnicas de Deep Learning (DL). Específicamente, se implementa una arquitectura de Red Neuronal Convolutiva (CNN) para la clasificación multiclase de imágenes. El modelo recibe como entrada radiografías de tórax (X) y produce como salida predicciones de clase ($Y \in \{\text{COVID-19, Neumonía, Normal}\}$). El aprendizaje se realiza a través de transfer learning, aprovechando características pre-entrenadas en ImageNet (base de datos de millones de imágenes naturales) y ajustando los pesos de la red para la tarea específica de clasificación radiológica.

Arquitecturas CNN Consideradas:

ResNet50 (He et al., 2015): Red residual con 50 capas que introduce conexiones saltadas (skip connections) para facilitar el flujo de gradientes. Excelente generalización y entrenamiento estable.

VGG16 (Simonyan & Zisserman, 2014): Arquitectura simple con 16 capas convolucionales y pequeños kernels 3×3 . Base sólida para transfer learning, aunque requiere más memoria.

DenseNet121 (Huang et al., 2017): Conexiones densas entre capas (cada capa conecta con todas las posteriores). Reduce gradiente desvaneciente y mejora propagación de información. Particularmente efectiva en imágenes médicas.

InceptionV3 (Szegedy et al., 2016): Módulos Inception que aplican convoluciones paralelas de diferentes tamaños. Captura características multi-escala; entrada de 299×299 píxeles.

Estrategia de Transfer Learning:

Se cargan los pesos pre-entrenados de ImageNet para la base convolutiva de cada arquitectura. Se congela la mayoría de capas convolucionales (para preservar características genéricas aprendidas en ImageNet) y se reemplaza la última capa de clasificación (softmax de 1000 clases) por un adaptador con capas personalizadas: capas densas con regularización L2, dropout (tasa 0.5), batch normalization, y una salida de 3 unidades con activación softmax para las tres clases del problema. El

ajuste fino (fine-tuning) se realiza en dos fases: primero se entrena solo el adaptador con base congelada, posteriormente se descongelan las últimas capas convolucionales y se entrena todo el modelo con learning rate reducido ($1e-5$).

Función de Pérdida y Optimizador:

- Función de Pérdida: Categorical Crossentropy (para clasificación multiclase de imágenes etiquetadas mutuamente excluyentes).
- Optimizador: Adam (adaptive learning rate, momento adaptativo). Parámetros: learning rate inicial = 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-7$.

Regularización y Técnicas Anti-Sobreaajuste:

- L2 Regularization (weight decay = 0.0001): Penaliza pesos grandes para mejorar generalización.
- Dropout (tasa = 0.5): Desactiva aleatoriamente neuronas durante el entrenamiento para evitar co-adaptación.
- Data Augmentation: Transformaciones aleatorias en tiempo de entrenamiento amplían la variabilidad.
- Early Stopping (patience = 10): Detiene el entrenamiento si val_loss no mejora en 10 épocas.

```
from tensorflow.keras import layers, models

# Usamos MobileNetV2 como base (Transfer Learning)
base_model = tf.keras.applications.MobileNetV2(input_shape=(224, 224, 3), include_top=False, weights='imagenet')
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax') # CAMBIO CLAVE: 3 clases
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

Total params: 2,422,339 (9.24 MB)

Trainable params: 164,355 (642.01 KB)

Non-trainable params: 2,257,984 (8.61 MB)

4.2 Desarrollo e Implementación del Modelo

Entorno de Desarrollo:

- Lenguaje: Python 3.10+
- Framework: TensorFlow 2.x con Keras API
- GPU: NVIDIA Tesla T4 (Google Colab Pro) con 16 GB VRAM
- Notebook: Jupyter en Google Colab (sin requerimientos de instalación local)
- Bibliotecas auxiliares: NumPy, Pandas, Matplotlib, Scikit-learn, OpenCV, Pillow

Fases de Implementación:

Fase 1: Preparación de Datos

- Descarga de dataset desde Kaggle usando Kaggle API.
- Exploración y análisis descriptivo: conteo de imágenes por clase, visualización de ejemplos, estadísticas de resolución.
- Split del dataset en train/val/test respetando la proporción 80/10/10 y balanceando clases.

Fase 2: Construcción del Pipeline de Preprocesamiento

- Implementación de ImageDataGenerator con aumento de datos.
- Creación de flow_from_directory para lectura eficiente desde disco.
- Validación visual de transformaciones aplicadas.

Fase 3: Construcción de Modelos

- Para cada arquitectura (ResNet50, VGG16, DenseNet121, InceptionV3): cargar pesos preentrenados, congelar base, agregar capas personalizadas.
- Compilar modelo con optimizer Adam, loss categorical_crossentropy, metrics [accuracy, precision, recall].
- Definir callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint.

Fase 4: Entrenamiento y Validación

- Entrenamiento con train_generator y validation_generator, epochs = 50 (con early stopping).
- Monitoreo de métricas en tiempo real: loss, accuracy, val_loss, val_accuracy por época.
- Visualización de curvas de aprendizaje (loss y accuracy vs. época).

Fase 5: Evaluación en Conjunto de Pruebas

- Predicción en test_generator (sin data augmentation).
- Cálculo de métricas: Accuracy, Precision, Recall, F1-score por clase, Matriz de Confusión, Curvas ROC.
- Comparación de rendimiento entre arquitecturas.

Fase 6: Interpretabilidad (Grad-CAM)

- Implementación de Gradient-weighted Class Activation Mapping (Grad-CAM).
- Visualización de regiones de interés (heatmaps) que el modelo utiliza para cada predicción.
- Validación clínica: verificar que las regiones destacadas corresponden a patologías conocidas.

```
# Definimos 10 épocas de entrenamiento (ciclos completos de aprendizaje)
EPOCHS = 10

print("Iniciando el proceso de adquisición de Experiencia (E) con Aprendizaje Supervisado...")
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
print("\nEntrenamiento finalizado con éxito!")
```

```
Iniciando el proceso de adquisición de Experiencia (E) con Aprendizaje Supervisado...
Epoch 1/10
153/153 ————— 103s 593ms/step - accuracy: 0.9028 - loss: 0.2575 - val_accuracy: 0.9549 - val_loss: 0.1377
Epoch 2/10
153/153 ————— 66s 429ms/step - accuracy: 0.9598 - loss: 0.1123 - val_accuracy: 0.9631 - val_loss: 0.1064
Epoch 3/10
153/153 ————— 69s 448ms/step - accuracy: 0.9705 - loss: 0.0846 - val_accuracy: 0.9598 - val_loss: 0.1151
Epoch 4/10
153/153 ————— 72s 473ms/step - accuracy: 0.9744 - loss: 0.0674 - val_accuracy: 0.9672 - val_loss: 0.0836
Epoch 5/10
153/153 ————— 66s 433ms/step - accuracy: 0.9815 - loss: 0.0515 - val_accuracy: 0.9664 - val_loss: 0.0834
Epoch 6/10
153/153 ————— 69s 458ms/step - accuracy: 0.9828 - loss: 0.0529 - val_accuracy: 0.9721 - val_loss: 0.0748
Epoch 7/10
153/153 ————— 66s 428ms/step - accuracy: 0.9861 - loss: 0.0361 - val_accuracy: 0.9598 - val_loss: 0.0877
Epoch 8/10
153/153 ————— 68s 442ms/step - accuracy: 0.9869 - loss: 0.0333 - val_accuracy: 0.9688 - val_loss: 0.0727
Epoch 9/10
153/153 ————— 73s 477ms/step - accuracy: 0.9873 - loss: 0.0347 - val_accuracy: 0.9729 - val_loss: 0.0786
Epoch 10/10
153/153 ————— 74s 481ms/step - accuracy: 0.9916 - loss: 0.0287 - val_accuracy: 0.9785 - val_loss: 0.0742

Entrenamiento finalizado con éxito!
```

```
import matplotlib.pyplot as plt

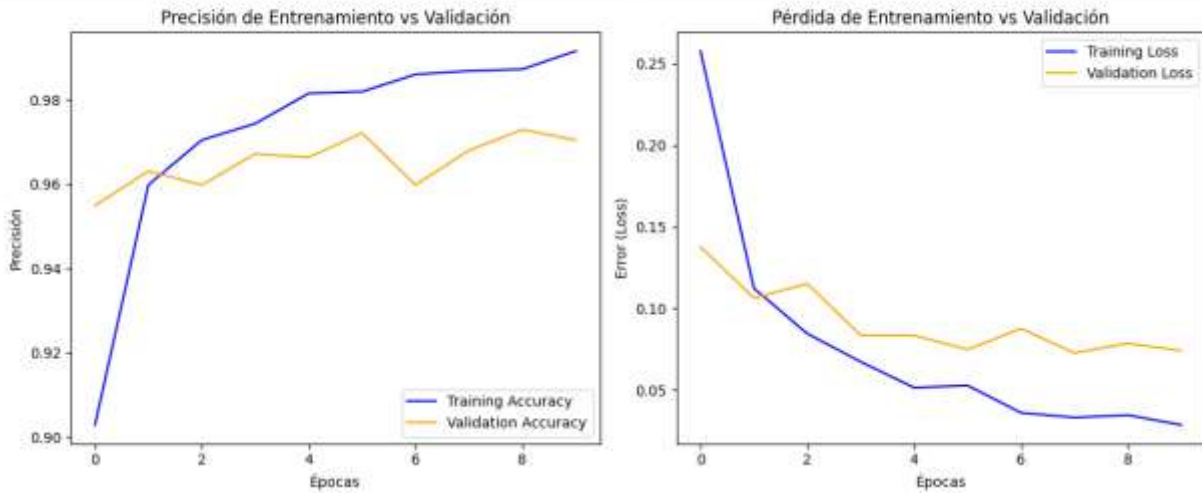
# Extraemos los datos del historial de entrenamiento
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(EPOCHS)

# Dibujamos las gráficas de rendimiento metodológico
plt.figure(figsize=(12, 5))

# Gráfica 1: Precisión (Accuracy)
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy', color='blue')
plt.plot(epochs_range, val_acc, label='Validation Accuracy', color='orange')
plt.title('Precisión de Entrenamiento vs Validación')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend(loc='lower right')

# Gráfica 2: Pérdida (Loss)
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss', color='blue')
plt.plot(epochs_range, val_loss, label='Validation Loss', color='orange')
plt.title('Pérdida de Entrenamiento vs Validación')
plt.xlabel('Épocas')
plt.ylabel('Error (Loss)')
plt.legend(loc='upper right')
```

```
plt.tight_layout()
plt.show()
```



Métricas de Evaluación:

Se utilizarán las siguientes métricas para evaluar el rendimiento de los modelos en el conjunto de pruebas: Accuracy (exactitud global), Precision (tasas de falsos positivos), Recall/Sensibilidad (tasas de falsos negativos), F1-Score (media armónica), Área Bajo la Curva ROC (AUC-ROC), Matriz de Confusión con normalización, y Curvas de Precisión-Recall por clase. Especial énfasis se pone en Recall para COVID-19 (minimizar falsos negativos) y visualización de Grad-CAM para interpretabilidad clínica.

REFERENCIAS BIBLIOGRÁFICAS

- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition.
- Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. arXiv preprint.
- Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv preprint arXiv:1409.1556.
- Szegedy, C., Vanhoucke, V., Ioffe, S., et al. (2016). Rethinking the Inception Architecture for Computer Vision. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV).
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Chollet, F., et al. (2015-2023). Keras: Deep Learning API. <https://keras.io/>
- Abadi, M., et al. (2015). TensorFlow: A System for Large-Scale Machine Learning. In 12th USENIX Symposium on Operating Systems Design and Implementation.
- Kaggle Dataset: COVID-19 Radiography Database.
<https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database>